

DOSSIER DE PROJET

JWE — Jurassic World Enclos

Candidat : Ivann René Denis RAMBAUD

Titre professionnel visé : Développeur Web et Web Mobile

Nature du projet : Projet réalisé pendant la formation, hors période en entreprise

Sommaire

1. Présentation du projet	3
1.1 Contexte et origine du besoin	3
1.2 Objectifs du projet	4
1.3 Spécificités techniques et fonctionnelles	4
2. Analyse et conception	5
2.1 Étude des besoins fonctionnels et techniques	5
2.2 Modélisation des données	6
2.3 Choix technologiques et outils	7
2.4 Maquettes et wireframes	8
3. Développement	9
3.1 Mise en place de l'environnement de développement	9
3.2 Modèle de données et relations aime / n'aime pas	10
3.3 Règles de compatibilité et cas particulier des charognards	11
3.4 API de compatibilité consommée en AJAX	12
3.5 Formulaire dynamiques et Select2	13
3.6 Filtrage combiné côté client	14
3.7 Veille sécurité et correction associée	15
3.8 Difficultés rencontrées : cinq incidents de déploiement	16
4. Tests et validation	17
4.1 Méthodologie de test	17
4.2 Jeu d'essai	18
5. Documentation	19
5.1 Documentation utilisateur	19
5.2 Documentation technique	20
6. Bilan personnel	21
Annexes	22

1. Présentation du projet

Glossaire des termes du jeu

Quelques termes propres au jeu Jurassic World Evolution 2, utilisés tout au long de ce dossier :

Terme	Définition
Enclos	Espace clos dans lequel un ou plusieurs dinosaures peuvent être placés ensemble.
Attrait	Score individuel d'une espèce influant sur les revenus touristiques du parc.
Génome	Ensemble de données propre à une espèce, débloqué en jeu par extraction depuis un fossile.
Affinité (aime / n'aime pas)	Préférence positive ou négative d'une espèce envers une autre, influant sur la compatibilité en enclos.
Charognard	Sous-catégorie d'espèces se nourrissant de cadavres plutôt que de proies vivantes, avec des règles de cohabitation particulières.

1.1 Contexte et origine du besoin

Dans Jurassic World Evolution 2, chaque parc est noté sur une échelle de 1 à 5 étoiles. Cette note dépend notamment des revenus générés par le parc — billetterie et commerces — qui sont eux-mêmes directement liés à l'attrait touristique des enclos. Ce score d'attrait varie énormément d'une espèce à l'autre : à titre d'exemple, un Mosasaure rapporte environ 2500 points d'attrait quand un Struthiomimus n'en rapporte qu'une vingtaine. Optimiser la composition de ses enclos a donc un impact direct sur les revenus et la note globale du parc, d'autant plus que le jeu propose des défis à accomplir en temps limité, où il faut atteindre 5 étoiles avant l'expiration d'un chronomètre.

Cependant, en jeu, les informations d'une espèce (régime alimentaire, compatibilités, besoins en végétation) ne sont accessibles qu'après avoir envoyé une expédition sur le terrain pour en récupérer des fossiles, puis extrait son génome en laboratoire. Le pourcentage du génome doit être supérieur ou égal à 50 — un processus qui prend du temps de jeu. Découvrir après extraction qu'une espèce n'est finalement pas compatible avec celles déjà présentes dans l'enclos fait perdre un temps précieux, en particulier dans le cadre d'un défi chronométré.

TRICÉRATOPS

ATTRAIT : 108

4



BESOINS DES DINOSAURES



FIBRES AU RAS DU SOL

48 %



FORÊT

48 %



EAU

5 %

POPULATION MINIMUM

2



AIME



ORNITHOMIMOSAURIEN



CÉRATOPSIEN



ANKYLOSAURIDÉ



HADROSAURIDÉ



CHAROIGNARD



N'AIME PAS



STYRACOSAURE



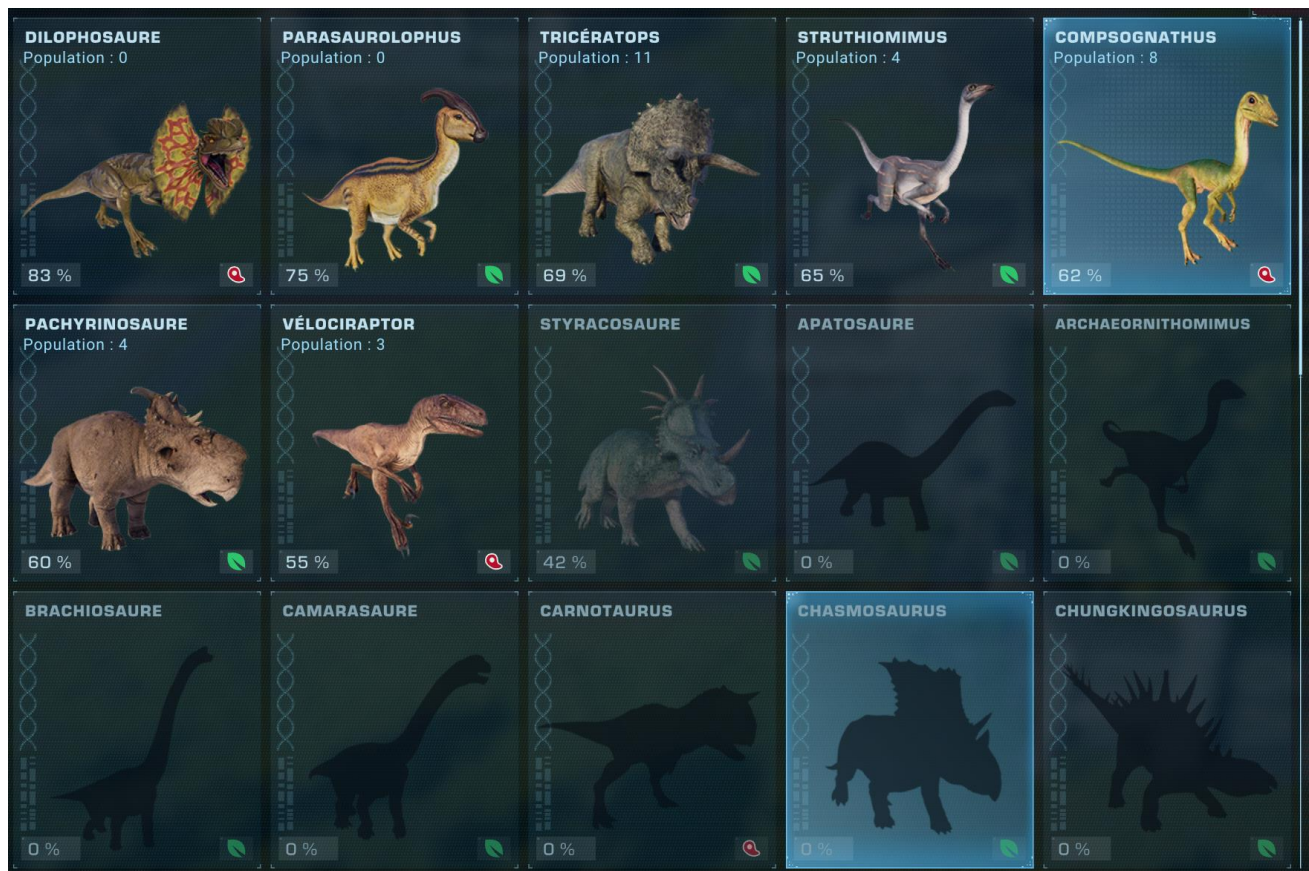
TOROSAURUS



SINOCÉRATOPS

CONDITIONS

1:15



J'ai développé ce simulateur en autoformation personnelle, en dehors de tout cadre scolaire, pour répondre à ce besoin : disposer d'un outil externe qui centralise les données de compatibilité et calcule instantanément, à partir d'une sélection de dinosaures, les autres espèces qui peuvent rejoindre le même enclos — afin d'optimiser mes enclos sans recherche manuelle ni perte de temps en jeu.

Cette section détaille les objectifs fonctionnels visés par le projet, avant de préciser en 1.3 les contraintes techniques qui en découlent et le périmètre finalement retenu.

1.2 Objectifs du projet

- Vérifier la compatibilité entre dinosaures : le cœur du besoin, permettre de savoir si plusieurs espèces peuvent cohabiter dans un même enclos.
- Sélection dynamique : pouvoir choisir un ou plusieurs dinosaures dans une liste, avec une fonction de recherche/filtrage, et voir cette liste se mettre à jour pour ne plus afficher que les espèces compatibles avec la sélection en cours.
- Gestion des fiches dinosaures : pouvoir ajouter, modifier et supprimer des dinosaures et leurs caractéristiques (type, famille, régime alimentaire, affinités). N'ayant pas trouvé d'API publique donnant accès aux données du jeu, j'ai dû prévoir une saisie manuelle de chaque espèce au fur et à mesure de mes découvertes en jeu.

Cahier des charges fonctionnel

Pour cadrer le développement, les trois objectifs de la section précédente ont été reformulés en exigences fonctionnelles numérotées, utilisées ensuite comme grille de vérification en fin de projet :

- EF1 — Un utilisateur peut sélectionner un ou plusieurs dinosaures et obtenir la liste des espèces compatibles avec l'ensemble de la sélection, mise à jour sans rechargement de page.
- EF2 — Un utilisateur peut filtrer la liste des dinosaures affichés par type et par régime alimentaire, et rechercher par nom.
- EF3 — Un utilisateur peut créer, modifier et supprimer une fiche dinosaure, y compris ses affinités et incompatibilités.
- EF4 — Le système applique de façon cohérente les règles de compatibilité par environnement, par régime alimentaire (avec l'exception charognard), et par déclaration individuelle, quel que soit l'ordre de comparaison des dinosaures.
- EF5 — Les médias (photos de dinosaures) restent accessibles après un redéploiement de l'application en production.

EF1 à EF4 sont directement vérifiées par le jeu d'essai de la section 4.2 ; EF5 découle du choix technique documenté en section 2.3 (Cloudinary) et a été validée empiriquement lors des incidents de déploiement de la section 3.8.

1.3 Spécificités techniques et fonctionnelles

Fonctionnellement, l'application repose sur un modèle de données central (le dinosaure) enrichi de plusieurs types de relations — affinités individuelles, incompatibilités de type et de famille — combinées par un moteur de règles pour produire, à la demande, la liste des espèces compatibles avec une sélection donnée. Techniquement, elle est bâtie sur Django (voir section 2.3 pour la justification des choix), avec un stockage des médias externalisé sur Cloudinary pour tenir compte des contraintes d'hébergement (voir section 3.8).

Le périmètre du projet a évolué en cours de route. Initialement, je comptais me concentrer uniquement sur les herbivores : ce sont les espèces les plus nombreuses dans le jeu, avec pour la plupart un score d'attrait individuel faible, ce qui rend leur mélange en enclos particulièrement nécessaire pour optimiser l'espace et les revenus. J'ai ensuite étendu l'application à l'ensemble des types de dinosaures (carnivores, piscivores, reptiles marins et volants) pour en faire un outil complet.

Le projet reste volontairement limité à un usage personnel : pas de gestion multi-utilisateurs, pas d'authentification (voir la justification en section 3.7, veille sécurité). Le maquettage de la refonte visuelle de l'accueil (section 2.4) reste également un prototype non intégré à ce jour à l'application en production — limite assumée et documentée plutôt que présentée comme terminée.

Contraintes non fonctionnelles

Deux contraintes non fonctionnelles ont pesé sur les choix techniques au-delà du strict fonctionnel. D'une part, la réactivité de l'interface : le recalcul de compatibilité devait rester perceptible comme instantané pour rester utile en situation de jeu chronométrée (section 1.1), ce qui a orienté le choix vers une API légère plutôt qu'un traitement plus coûteux côté serveur. D'autre part, la robustesse face aux erreurs de saisie : la fonction `verifier_compatibilite()` doit toujours retourner un résultat exploitable même si une fiche dinosaure est incomplète (famille non renseignée, par exemple), ce que garantissent les vérifications `and dino2.famille` avant chaque comparaison de catégorie dans le code (section 3.3).

1.4 Compétences mobilisées

Le projet mobilise des compétences réparties sur les deux CCP du titre visé :

CCP1 — Développer la partie front-end d'une application web ou web mobile sécurisée

- Maquetter des interfaces utilisateur : maquette interactive de refonte de l'accueil (section 2.4).
- Réaliser des interfaces utilisateur statiques : templates Django (HTML/CSS), adaptation responsive de l'ensemble du site.
- Développer la partie dynamique des interfaces : mise à jour AJAX du simulateur d'enclos, recherche dynamique (`django-select2`), voir sections 3.4 à 3.6.

CCP2 — Développer la partie back-end d'une application web ou web mobile sécurisée

- Mettre en place une base de données relationnelle : modélisation des dinosaures et de leurs relations, migrations Django, passage SQLite vers PostgreSQL (sections 2.2 et 2.3).
- Développer des composants d'accès aux données : ORM Django, requêtes de filtrage dans `verifier_compatibilite()` et `get_compatibles()` (section 3.3-3.4).
- Développer des composants métier côté serveur : logique de compatibilité, vues Django, API JSON (sections 3.2 à 3.4).

Deux compétences supplémentaires, hors périmètre strict des deux CCP ci-dessus, ont également été mises en œuvre : installer et configurer son environnement de travail (section 3.1), et documenter le déploiement d'une application dynamique (sections 3.8 et 5.2).

2. Analyse et conception

Cette section retrace le travail d'analyse mené avant la phase de développement proprement dite, dont les résultats concrets (modèle de données, choix technologiques, maquette) sont détaillés dans les sous-sections suivantes.

2.1 Étude des besoins fonctionnels et techniques

Avant de coder, j'ai listé les opérations que le jeu ne permet pas de faire directement et que l'outil devait couvrir : centraliser les fiches d'espèces au fur et à mesure de leur découverte, exprimer trois familles de

règles de compatibilité (par type d'environnement, par affinité individuelle, par catégorie), et restituer un résultat de compatibilité instantané pour une sélection multiple plutôt que pour une seule paire de dinosaures à la fois — c'est ce dernier point qui a le plus structuré l'architecture technique, puisqu'il impose de croiser chaque candidat avec l'ensemble des dinosaures déjà choisis (voir section 3.4).

Sur le plan technique, l'analyse a fait ressortir trois contraintes : la nécessité d'une interface qui se met à jour sans rechargement de page (d'où le choix d'une API JSON consommée en AJAX plutôt que des formulaires classiques), la nécessité de modéliser des relations non symétriques entre dinosaures (un dinosaure peut détester une espèce sans que la réciproque soit vraie, voir section 3.2), et la contrainte de stockage des médias imposée par l'hébergement choisi en production (voir section 3.8).

Alternatives envisagées

Deux choix d'architecture ont été discutés avant d'être tranchés. D'abord, la mise à jour de la liste des compatibles aurait pu se faire par rechargement de page (formulaire classique en POST) plutôt que par API JSON en AJAX : j'ai écarté cette option parce que le cœur du besoin (section 1.2) est justement de pouvoir tester plusieurs combinaisons rapidement sans perdre le fil, ce qu'un rechargement complet à chaque sélection aurait pénalisé en confort d'usage, même si l'implémentation aurait été plus simple côté serveur.

Ensuite, pour les champs de sélection multiple (affinités, incompatibilités, nourriture), j'ai choisi django-select2 plutôt qu'un simple <select multiple> HTML natif. Un menu déroulant natif devient difficile à utiliser dès qu'il contient plus d'une dizaine d'options : pas de recherche par le texte, sélection multiple peu lisible. Select2 ajoute une barre de recherche et un affichage sous forme d'étiquettes, ce qui reste utilisable même quand la base de dinosaures grossit — un choix pensé pour la maintenance à long terme du projet plutôt que pour son état actuel, encore limité en nombre de fiches.

2.2 Modélisation des données

Le modèle central de l'application est `Dinosaure`, complété par un modèle `Nourriture` en relation many-to-many (un dinosaure peut avoir plusieurs régimes alimentaires possibles, une nourriture peut être partagée par plusieurs espèces) :

```
class Nourriture(models.Model):
    nom = models.CharField(max_length=100, unique=True)

class Dinosaure(models.Model):
    TYPE_CHOICES = [
        ('Herbivore', 'Herbivore'), ('Carnivore', 'Carnivore'),
        ('Piscivore', 'Piscivore'), ('Reptile marin', 'Reptile marin'),
        ('Reptile volant', 'Reptile volant'),
    ]
    FAMILLE_CHOICES = [
        ('Sauropodes', 'Sauropodes'), ('Pachycéphalosauridé', '...'),
        ('Hadrosauridé', '...'), ('Ankylosauridé', '...'),
        ('Cératopsien', '...'), ('Ornithomimosaurien', '...'),
        ('Charognard', 'Charognard'), ('Hybrides', 'Hybrides'),
    ]
    nom = models.CharField(max_length=100)
```

```

type = models.CharField(max_length=50, choices=TYPE_CHOICES)
famille = models.CharField(max_length=100, choices=FAMILLE_CHOICES,
                           blank=True, null=True)
nourriture = models.ManyToManyField(Nourriture, blank=True)
image = CloudinaryField('image', blank=True, null=True)

aime = models.ManyToManyField('self', symmetrical=False, blank=True,
                             related_name='aimé_par')
naime_pas = models.ManyToManyField('self', symmetrical=False,
                                   blank=True, related_name='détesté_par')

naime_pas_types = models.CharField(max_length=200, blank=True)
naime_pas_familles = models.CharField(max_length=200, blank=True)

```

Deux choix de modélisation méritent d'être justifiés. D'abord, le champ `type` est une simple énumération (`TYPE\_CHOICES`) plutôt qu'un modèle séparé : le nombre de types est fixe et connu à l'avance, une table dédiée aurait ajouté une jointure sans bénéfice réel. À l'inverse, `famille` a une utilité qui dépasse l'affichage : les incompatibilités de catégorie (section 3.3) et la règle des charognards s'appuient dessus, donc autant qu'elle vive dans le même modèle plutôt que dans une table séparée qu'il faudrait joindre à chaque vérification.

Ensuite, les incompatibilités de type et de famille (`naime\_pas\_types`, `naime\_pas\_familles`) sont stockées comme des chaînes de caractères séparées par des virgules plutôt que comme de vraies relations Many-to-Many. C'est un compromis assumé : une vraie relation M2M aurait été plus « propre », mais ces deux champs ne référencent pas d'autres dinosaures, seulement des catégories (des chaînes parmi `TYPE\_CHOICES`/`FAMILLE\_CHOICES`), ce qui aurait demandé de créer deux modèles supplémentaires uniquement pour porter cette liste. J'ai préféré ce raccourci pragmatique, au prix d'un parsing manuel (`.split(',')`) au moment de la vérification (voir section 3.3).

Schéma conceptuel simplifié

```

Nourriture (nom)
  ^
  | 0..N (ManyToMany)
Dinosaure (nom, type, famille, image)
  |--- aime          --> Dinosaure (ManyToMany, symmetrical=False)
  |--- naime_pas      --> Dinosaure (ManyToMany, symmetrical=False)
  |--- naime_pas_types (CharField, liste de TYPE_CHOICES séparés par virgules)
  |--- naime_pas_familles (CharField, liste de FAMILLE_CHOICES séparés par virgules)

```

Ce schéma résume la section précédente : un seul modèle central (Dinosaure) porte à la fois ses attributs descriptifs et l'ensemble de ses relations de compatibilité, qu'elles soient de vraies relations Django (aime, naime_pas, nourriture) ou des champs texte faisant office de relations simplifiées vers des catégories (naime_pas_types, naime_pas_familles).

2.3 Choix technologiques et outils

Le choix du langage Python, et par extension du framework Django, découle de mon niveau de confort avec ce langage : c'est celui que je maîtrise le mieux. Django présentait un avantage supplémentaire pour un projet solo : sa base de données SQLite est gérée nativement, sans nécessiter l'installation et la configuration

d'un serveur de base de données externe comme MySQL. Cette centralisation limitait les sources de bugs liées à la configuration d'un environnement de base de données séparé.

Le passage à PostgreSQL et à Cloudinary en production découle d'une contrainte technique découverte au moment du déploiement sur Render : la base de données PostgreSQL fournie gratuitement par Render n'est pas permanente — sur le plan gratuit, elle est automatiquement supprimée au bout d'une durée limitée (environ un à deux mois), ce qui a d'ailleurs causé l'incident de déploiement décrit plus loin (base de données expirée). Cloudinary a été retenu en parallèle comme service de stockage externe persistant pour les médias (images des dinosaures), sans cette limite de durée.

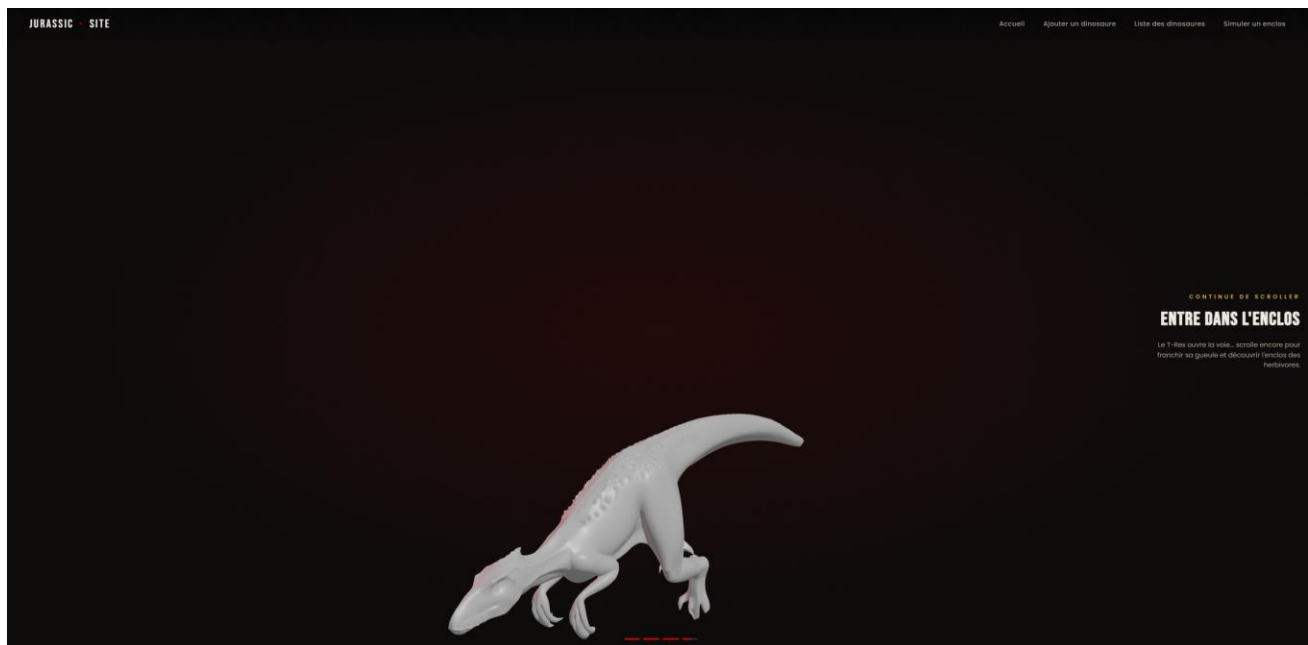
Stack technique retenue : Django 5.1, SQLite en développement / PostgreSQL en production, JavaScript natif (Fetch API) côté client, django-select2 pour les champs de sélection multiple avec recherche, Cloudinary pour le stockage des médias, déploiement sur Render via Gunicorn et WhiteNoise pour les fichiers statiques.

2.4 Maquettes et wireframes

Le site actuel n'a pas fait l'objet d'une maquette avant développement : le design a été construit directement en HTML/CSS, au fil de l'eau. En préparant ce dossier, j'ai voulu corriger cette lacune en amont du développement, cette fois, plutôt qu'après coup.

J'ai construit une maquette interactive (fichier HTML/CSS/JavaScript autonome) pour explorer une nouvelle direction visuelle et une nouvelle séquence de navigation pour la page d'accueil, transition en forme de mâchoire qui se referme sur l'écran au scroll, puis scène 3D (Three.js, `GLTFLoader`, modèle au format `.glb`) où un dinosaure tourne progressivement à mesure du défilement, avant de révéler les cartes de navigation vers les autres pages du site (accueil, liste des dinosaures, ajout d'un dinosaure). L'ensemble de la séquence — rotation du modèle 3D, apparition progressive des éléments d'interface — sera piloté par une fonction qui convertit la position de scroll en un pourcentage de progression borné entre 0 et 1, avec une fonction de easing pour adoucir les transitions.





Cette maquette reste un prototype de direction visuelle, testé isolément et non encore intégré à l'application Django en production ; le modèle 3D utilisé est un modèle générique en attendant un modèle dédié aux dinosaures du site. Le schéma formel de l'enchaînement des écrans et la vérification de l'accessibilité restent à produire — un travail que j'assume comme non terminé à ce jour, à poursuivre après l'obtention du titre.

Sources d'inspiration

La direction « liquid glass » (panneaux translucides, flou d'arrière-plan) et la logique de scène 3D pilotée par le scroll s'appuient sur des références que j'ai observées avant de me lancer dans la maquette, plutôt que d'être improvisées directement en code. En relisant ce prototype pour ce dossier, j'ai remis en question cette direction : je ne souhaite plus m'orienter vers l'effet liquid glass, mais plutôt vers un aspect plus premium pour la refonte de l'accueil. Les deux références suivantes, observées avant de retravailler cette direction, orientent la suite que je compte donner à la maquette plutôt que la version déjà prototypée ci-dessus.



<https://www.igloo.inc/>



<https://labs.noomoagency.com/>

Je souhaite reprendre le principe de la méduse pour l'idée d'une créature qui tourne autour de l'utilisateur sur la frame principale de mon accueil. Pour la deuxième frame, je m'inspire de cet igloo : le but est d'obtenir un environnement interactif donnant l'impression d'y être, avec certains éléments qui réagissent au survol de la souris.

3. Développement

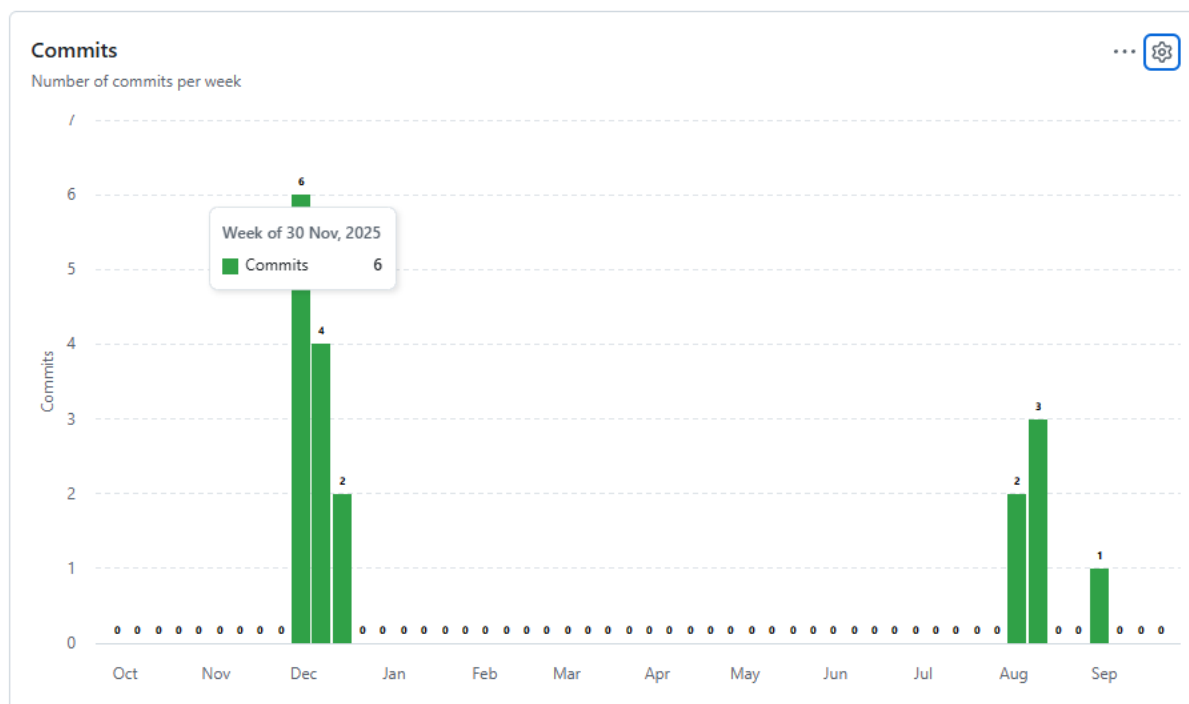
3.1 Mise en place de l'environnement de développement

Avant de commencer le développement, j'ai créé un environnement virtuel Python (`venv`) dédié au projet, initialisé un dépôt Git local puis créé le dépôt distant sur GitHub (`Ivann212/JWE`) pour versionner mon code. Les dépendances sont listées dans `requirements.txt`, régénéré à chaque ajout de bibliothèque avec `pip freeze`. Les informations sensibles (clé secrète Django, identifiants Cloudinary) sont séparées du code source versionné via un fichier `.env` local, non suivi par Git, lu au démarrage grâce à `python-dotenv`.

```
git clone https://github.com/Ivann212/JWE.git
cd JWE
python -m venv venv
venv\Scripts\Activate.ps1      # Windows
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

- └─ jurassic_site
 - > __pycache__
 - > migrations
 - > static
 - > templates
 - + __init__.py
 - + admin.py
 - + asgi.py
 - + forms.py
 - + models.py
 - + settings.py
 - + tests.py
 - + urls.py
 - + views.py
 - + wsgi.py
 - > media
 - > venv
 - ⚙ .env
 - 📄 .gitignore
 - 📄 db.sqlite3
 - + manage.py
 - 📖 README.md
 - 📄 requirements.txt

Commits over the last year of Ivann212/JWE



Le fichier `.gitignore` du dépôt exclut explicitement `venv/`, `__pycache__`, `db.sqlite3`, `.env` et les fichiers média locaux : l'objectif est qu'un environnement de développement propre se reconstruise entièrement à partir de `requirements.txt` et des migrations Django versionnées, sans dépendre d'un état local qui ne serait pas reproductible d'une machine à l'autre — c'est cette même logique de reproductibilité qui sous-tend la procédure de déploiement décrite en section 5.2.

3.2 Modèle de données et relations aime / n'aime pas

```
aime = models.ManyToManyField('self', symmetrical=False, blank=True,
related_name='aimé_par')
naime_pas = models.ManyToManyField('self', symmetrical=False, blank=True,
related_name='détesté_par')
```

J'ai modélisé les relations ``aime`` et ``naime_pas`` comme des ``ManyToManyField`` vers le modèle lui-même, avec ``symmetrical=False``. Ce choix évite qu'une déclaration dans un sens duplique automatiquement la donnée dans l'autre sens en base : je ne saisis la relation qu'une seule fois. En contrepartie, la fonction ``verifier_compatibilite()`` interroge explicitement les deux sens de la relation à chaque vérification, ce qui garantit que le filtrage des dinosaures compatibles reste cohérent quel que soit l'ordre dans lequel les espèces sont ajoutées à l'enclos — sans avoir besoin de stocker deux fois la même information.

Exemple concret

Pour illustrer le fonctionnement de ces relations : si `Diplodocus.aime.add(Gallimimus)` est enregistré, alors `Diplodocus.aime.all()` contient `Gallimimus`, mais `Gallimimus.aime.all()` reste vide — c'est le sens direct de la relation, cohérent avec ``symmetrical=False``. En revanche, grâce au `related_name='aimé_par'` défini sur le champ, `Gallimimus.aimé_par.all()` permet de retrouver `Diplodocus` sans requête sup-

plémentaire : Django expose automatiquement la relation inverse sous ce nom. C'est ce mécanisme que `verifier_compatibilite()` exploite indirectement en testant l'affinité dans les deux sens (`dino1.aime.filter(id=dino2.id)` puis `dino2.aime.filter(id=dino1.id)`) plutôt que de compter sur une seule direction.

3.3 Règles de compatibilité et cas particulier des charognards

La fonction `verifier_compatibilite(dino1, dino2)` applique une succession de règles, dans un ordre précis, et retourne dès qu'une règle tranche (compatible ou non) :

- Règle d'environnement : les reptiles marins ne sont compatibles qu'entre eux (enclos aquatique), de même pour les reptiles volants (volière séparée) — ces deux règles sont évaluées en premier et court-circuitent toute la suite.
- Règle carnivore / herbivore : incompatibles entre eux, sauf exception (voir ci-dessous).
- Règle piscivore : incompatible avec les carnivores et les herbivores.
- Incompatibilités individuelles (`naime_pas`), puis de type (`naime_pas_types`), puis de famille (`naime_pas_familles`) — vérifiées dans les deux sens à chaque fois.
- Affinité positive (`aime`) : si aucune incompatibilité n'a été trouvée, une affinité déclarée rend explicitement la paire compatible avec un message dédié.

Une règle mérite une attention particulière car elle n'apparaissait pas dans les premières versions de cette documentation : les dinosaures de la famille « Charognard » (attribut `famille`) sont exemptés des règles de type carnivore/herbivore/piscivore. Cette exception m'a semblé nécessaire en relisant le comportement du jeu : un charognard, bien que classé « Carnivore » dans le type, se nourrit de cadavres plutôt que de proies vivantes et cohabite donc sans problème avec des herbivores dans Jurassic World Evolution 2.

```
est_charognard = dino1.famille == 'Charognard' or dino2.famille == 'Charognard'

if not est_charognard:
    # Les carnivores ne sont pas compatibles avec les herbivores
    if (dino1.type == 'Carnivore' and dino2.type == 'Herbivore') or \
        (dino1.type == 'Herbivore' and dino2.type == 'Carnivore'):
        return False, "Les carnivores et herbivores ne peuvent pas cohabiter"
    # ... (règles piscivore, identiques dans leur principe)
```

Ce drapeau `est_charognard` encadre uniquement les règles de type (carnivore/herbivore/piscivore) : les règles d'environnement (aquatique/aérien) et les incompatibilités individuelles ou de catégorie restent, elles, pleinement applicables à un charognard — c'est volontaire, l'exception ne doit couvrir que le régime alimentaire, pas le reste des contraintes.

3.4 API de compatibilité consommée en AJAX

```
for dino_enclos in dinos_selectionnees:
    est_compatible, raison = verifier_compatibilite(dino_enclos, dino)
    if not est_compatible:
        est_compatible_avec_tous = False
        raisons.append(raison)
        break # Pas besoin de vérifier les autres si déjà incompatible
```

Cette vue (``get_compatibles``) expose une API JSON consommée en AJAX par le front-end (fichier ``enclos.html``), qui permet de recalculer dynamiquement la liste des dinosaures compatibles à chaque sélection, sans recharger la page. Elle reçoit une liste d'identifiants séparés par des virgules, récupère les dinosaures correspondants, puis compare chaque dinosaure restant à l'ensemble des dinosaures déjà sélectionnés via ``verifier_compatibilite()``. Une optimisation logique est appliquée dans la boucle de comparaison : dès qu'un dinosaure candidat s'avère incompatible avec un seul des dinosaures déjà présents dans l'enclos, l'instruction ``break`` interrompt la boucle interne — inutile de vérifier sa compatibilité avec les autres puisqu'il est de toute façon exclu du résultat final.

En relisant cette vue, j'ai identifié un point de sécurité à corriger : le bloc ``except`` final renvoyait initialement le message d'erreur Python brut au client (``str(e)``), ce qui pouvait exposer des détails internes à l'application en cas d'erreur imprévue. J'ai corrigé ce point en renvoyant un message générique au client, tout en conservant la trace de l'erreur réelle côté serveur pour le débogage :

```
except Exception as e:
    print(e)
    return JsonResponse({'success': False, 'error': "Une erreur est survenue"}, status=500)
```

Exemple de réponse renvoyée par cette API pour une sélection de deux herbivores compatibles :

```
{
  "success": true,
  "dinos_selectionnes": [
    {"id": 3, "nom": "Tricératops", "type": "Herbivore", "famille": "Cératopsien", "image": "https://res.cloudinary.com/.../triceratops.jpg"},
    {"id": 7, "nom": "Edmontosaure", "type": "Herbivore", "famille": "Hadrosauridé", "image": "https://res.cloudinary.com/.../edmontosaure.jpg"}
  ],
  "compatibles": [
    {"id": 12, "nom": "Styracosaure", "type": "Herbivore", "famille": "Cératopsien", "image": "https://res.cloudinary.com/.../styracosaure.jpg", "raison": "Compatible"}
  ],
  "count": 1
}
```

Le front-end (`enclos.html`) consomme directement cette structure : `dinos_selectionnes` réaffiche l'enclos courant, `compatibles` régénère la grille de sélection, et `raison` est affiché sous chaque carte pour expliquer pourquoi un dinosaure a été retenu ou écarté — c'est ce même champ `raison` qui porte les messages définis dans `verifier_compatibilite()` (section 3.3), comme « Compatible » ou « X apprécie Y ».

Déroulé complet d'une sélection

Pour relier les sections précédentes entre elles, voici le déroulé complet d'une action de l'utilisateur sur la page «Simuler un enclos», du clic jusqu'à l'affichage du résultat :

- 1. L'utilisateur clique sur un dinosaure dans la liste (via le champ de recherche `Select2`, section 3.5) : son identifiant est ajouté côté client à la liste des dinosaures sélectionnés.

- 2. Le JavaScript du front-end construit l'URL `/api/compatibles/<ids>/` avec la liste d'identifiants séparés par des virgules, et déclenche une requête `fetch()` vers cette API.
- 3. Côté serveur, la vue `get_compatibles()` (section 3.4) récupère les dinosaures correspondants, puis compare chaque dinosaure restant de la base à l'ensemble de la sélection via `verifier_compatibilite()` (section 3.3), en s'arrêtant dès la première incompatibilité trouvée pour chaque candidat.
- 4. La vue renvoie un JSON structuré (`dinosaurs_selectionnes`, `compatibles`, `count` — voir l'exemple en section 3.4).
- 5. Le JavaScript reçoit la réponse et appelle `appliquerFiltres()` (section 3.6), qui recombine ce résultat de compatibilité avec les filtres de recherche et de régime alimentaire actifs pour décider, carte par carte, quels dinosaures rester affichés dans la grille.

Ce découpage en cinq étapes montre pourquoi une régression sur n'importe laquelle des fonctions citées (`verifier_compatibilite`, `get_compatibles`, `appliquerFiltres`) peut se traduire par un symptôme visible ailleurs dans la chaîne — c'est aussi pour cette raison que le test #14 (section 4.2), ajouté après coup sur une seule de ces fonctions, m'a semblé indispensable malgré son coût de rédaction limité.

3.5 Formulaires dynamiques et Select2

La saisie et la modification des fiches dinosaures reposent sur deux `ModelForm` distincts.

`DinosaureAddForm` couvre la création (champs de base uniquement), tandis que `DinosaureEditForm` ajoute la gestion des affinités et incompatibilités, avec des widgets `django-select2` pour transformer les listes déroulantes multiples en champs de recherche ergonomiques :

```
class DinosaureEditForm(forms.ModelForm):
    naime_pas_types_select = forms.MultipleChoiceField(
        choices=Dinosaure.TYPE_CHOICES, required=False,
        widget=forms.SelectMultiple(attrs={'class': 'select2-multiple'}))

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        if self.instance.pk:
            self.fields['aime'].queryset = Dinosaure.objects.exclude(pk=self.instance.pk)
            self.fields['naime_pas'].queryset =
Dinosaure.objects.exclude(pk=self.instance.pk)

    def save(self, commit=True):
        instance = super().save(commit=False)
        instance.naime_pas_types = ','.join(self.cleaned_data['naime_pas_types_select'])
        instance.naime_pas_familles =
','.join(self.cleaned_data['naime_pas_familles_select'])
        if commit:
            instance.save()
            self.save_m2m()
        return instance
```

Deux détails d'implémentation découlent directement des choix de modélisation de la section 2.2. D'abord, le formulaire exclut systématiquement le dinosaure en cours d'édition de ses propres listes `'aime'/'naime_pas' (.exclude(pk=self.instance.pk))` : sans cela, rien n'empêcherait techniquement un

dinosaure de se déclarer incompatible avec lui-même. Ensuite, comme les champs `naime\_pas\_types` et `naime\_pas\_familles` sont stockés en base comme de simples chaînes de caractères (section 2.2) et non comme de vraies relations, le formulaire doit les convertir manuellement dans les deux sens : `.split(',')` pour préremplir le formulaire à l'ouverture, `','.join(...)` pour les resérialiser dans `save()`.

3.6 Filtrage combiné côté client

```
function appliquerFiltres() {
  let idsEligibles = dinosEligibles === null
    ? Object.keys(dinosData).map(Number)
    : dinosEligibles.map(d => d.id);

  document.querySelectorAll('.dino-card').forEach(card => {
    const id = parseInt(card.dataset.id, 10);
    const dino = dinosData[id];
    const estEligible = idsEligibles.includes(id);
    const correspondRecherche = !termeRecherche ||
dino.nom.toLowerCase().includes(termeRecherche);
    const correspondNourriture = !nourritureChoisie ||
dino.nourriture.includes(nourritureChoisie);
    card.style.display = (estEligible && correspondRecherche && correspondNourriture) ?
'block' : 'none';
  });
}
```

En reprenant le simulateur d'enclos pour ce dossier, j'ai identifié trois limites par rapport à mon cahier des charges initial : la fonction de recherche annoncée n'avait finalement jamais été implémentée, il manquait un filtre par régime alimentaire (deux dinosaures peuvent être compatibles entre eux tout en nécessitant des végétations différentes, ce qui pose un problème d'espace disponible dans l'enclos), et un bug empêchait la liste de se réafficher correctement après avoir vidé l'enclos dans certains cas.

J'ai choisi de résoudre les trois problèmes ensemble plutôt que séparément, en centralisant toute la logique d'affichage dans une seule fonction, `appliquerFiltres()`, appelée à chaque fois qu'un des critères change (sélection dans l'enclos, recherche par nom, filtre de régime). Ce choix de centralisation a corrigé le bug de réaffichage sans code correctif dédié : l'ancienne implémentation modifiait l'état d'affichage de la grille à plusieurs endroits différents, et un de ces endroits ne réinitialisait pas correctement l'état précédent une fois l'enclos vidé. En ne calculant plus l'affichage qu'à un seul endroit, ce type d'incohérence devient structurellement impossible.

Enclos actuel

 amargasaurus Herbivore ✕

 chungkingosaurus Herbivore ✕

2

Dans l'enclos

22

Disponibles

Vider l'enclos

Ajouter un dinosaure (22 compatibles)

Rechercher un dinosaure par nom...

Tous les régimes alimentaires



archaeornithomimus
Herbivore
Ornithomimosaurien
Compatible



coelophysis
Carnivore
Charognard
Compatible



compsognathus
Carnivore
Charognard
Compatible



corythosaurus
Herbivore
Hadrosaunidé
Compatible



dracorex
Herbivore
Pachycéphalosauridé
Compatible



dryosaurus
Herbivore
Compatible



edmontosaurus
Herbivore



gallimimus
Herbivore



gigantspinosaurus
Herbivore



homalocephale
Herbivore



huayangosaurus
Herbivore



iguanodon
Herbivore

Enclos actuel



Aucun dinosaure dans l'enclos

Sélectionnez un dinosaure ci-dessous pour commencer

Sélectionnez votre premier dinosaure

Rechercher un dinosaure par nom...

Noix en hauteur



brachiosaure
Herbivore
Sauropodes



camarasaure
Herbivore
Sauropodes

Enclos actuel



Aucun dinosaure dans l'enclos
Sélectionnez un dinosaure ci-dessous pour commencer

Sélectionnez votre premier dinosaure

tyr

Viandes



tyrannosaure
Carnivore

3.7 Veille sécurité et correction associée

Recherche effectuée sur le moteur de recherche avec les mots-clés `Django DEBUG production security risk`, ayant mené à l'article « How a Simple Django Debug Setting Exposed User Passwords and Database Secrets » (Medium, Pankaj Kumar Yadav, source anglophone).

Cet article documente un cas réel où le paramètre `DEBUG = True`, laissé actif sur un serveur de production Django, a permis à n'importe quel visiteur non authentifié d'accéder aux requêtes SQL exécutées par l'application, à la structure complète de la base de données, ainsi qu'à des hachages de mots de passe utilisateurs — exposant l'application à un risque de craquage de mots de passe hors ligne et de compromission de comptes administrateurs.

Ce constat rejoint directement un problème identifié sur mon propre projet : mon application était déployée en production avec `DEBUG=True`, affichant à tout visiteur une page d'erreur Django complète en cas d'exception dès qu'une erreur survenait. J'ai corrigé ce point en configurant `DEBUG` via une variable d'environnement, avec une valeur par défaut à `False`.

Cette veille m'a également permis d'identifier une seconde faiblesse de mon application, restée en l'état à ce jour : l'absence de tout système d'authentification. N'importe quel visiteur connaissant l'URL du site peut actuellement ajouter, modifier ou supprimer des fiches dinosaures sans être connecté. Ce choix s'explique par la nature du projet : il s'agit d'un outil strictement personnel, mis en ligne dans le cadre de ma préparation au titre DWWM pour en faciliter la démonstration, et non d'une application destinée à un usage public multi-utilisateur. Une évolution possible, si l'application devait accueillir d'autres utilisateurs, serait de restreindre ces actions aux personnes authentifiées via le système d'authentification intégré à Django.

Recherche à partir d'un site anglophone

Après avoir découvert que mon application tournait avec `DEBUG=True` en production, j'ai voulu comprendre précisément la gravité et les mécanismes de ce risque avant de le corriger, au-delà du simple

constat empirique fait sur mon propre site. N'ayant pas trouvé de ressource francophone aussi précise et argumentée sur le sujet, j'ai consulté le même article technique en anglais.

Extrait original (anglais), tiré de la section « Root Cause Analysis » de l'article « How a Simple Django Debug Setting Exposed User Passwords and Database Secrets » (Pankaj Kumar Yadav, Medium) :

« This vulnerability exists because of a single configuration error. When `DEBUG = True` in Django: the debug toolbar (if installed) is active, detailed error pages with stack traces are shown, SQL queries are logged and displayed, and internal settings are exposed through the toolbar. In development, this is invaluable. In production, it's catastrophic. The fix is devastatingly simple: set `DEBUG = False`. Additionally, ensure the Django Debug Toolbar is only installed in development dependencies, and never included in production requirements. »

Ma traduction (réalisée sans traducteur automatique) :

« Cette vulnérabilité existe simplement à cause d'une petite erreur de configuration. Quand `DEBUG=True` est activé dans Django : la barre d'outils de debug (si installée) est active, les pages d'erreur détaillées avec la trace complète sont affichées, les requêtes SQL sont journalisées et affichées, et les paramètres internes de l'application sont accessibles depuis cette barre d'outils. En développement, c'est précieux. En production, c'est une catastrophe. La solution est d'une simplicité déconcertante : passer `DEBUG` à `False`. Il faut également s'assurer que la barre d'outils de debug de Django n'est installée qu'en développement, et jamais incluse dans les dépendances de production. »

Protections CSRF et XSS

Au-delà du point `DEBUG`/authentification traité en section 3.7, deux protections web standards sont actives sur le projet sans avoir demandé de configuration supplémentaire de ma part, ce qui méritait d'être compris plutôt que simplement constaté. Django insère automatiquement un jeton CSRF (Cross-Site Request Forgery) dans chaque formulaire via le tag `{% csrf_token %}` des templates, et rejette toute requête POST qui ne le présente pas — ce qui empêche un site tiers de soumettre, à l'insu de l'utilisateur, une requête de modification ou de suppression de dinosaure en se faisant passer pour lui.

De la même façon, le moteur de templates Django échappe par défaut toute variable affichée (échappement HTML automatique), ce qui protège contre les failles XSS (Cross-Site Scripting) : si un nom de dinosaure contenait par erreur des caractères spéciaux ou une tentative d'injection de script, il serait affiché tel quel comme texte plutôt qu'interprété comme du code par le navigateur. Ces deux protections n'ont demandé aucune ligne de code supplémentaire, mais les comprendre m'a semblé nécessaire pour ne pas les désactiver par inadvertance (par exemple en utilisant le filtre `|safe` sans réfléchir) lors d'une évolution future du projet.

Limite de performance identifiée

En relisant `get_compatibles()` (section 3.4) pour ce dossier, j'ai identifié une limite de performance non corrigée à ce jour : la boucle qui compare chaque dinosaure de la base à la sélection courante appelle `dino_enclos.naime_pas.all()` et `dino.aime.filter(...)` à l'intérieur d'une boucle Python, ce qui génère potentiellement une requête SQL par comparaison plutôt qu'une requête groupée. Avec le volume actuel de fiches (quelques dizaines), l'effet reste imperceptible, mais ce schéma dit «N+1 requêtes» deviendrait un vrai problème de performance si la base de dinosaures grossissait significativement. La correction

identifiée mais non encore implémentée consisterait à précharger les relations aime et n aime_pas de l'ensemble des dinosaures concernés en une seule requête, via `prefetch_related()`, avant d'entrer dans la boucle de comparaison.

3.8 Difficultés rencontrées : cinq incidents de déploiement

Après avoir développé JWE en local avec SQLite, j'ai préparé et documenté son déploiement en production sur Render, ce qui m'a demandé de rédiger la procédure de mise en ligne (variables d'environnement, commande de démarrage Gunicorn, migration vers PostgreSQL et Cloudinary) et de la faire évoluer au fil des incidents rencontrés :

- Base de données PostgreSQL expirée — le service refusait de démarrer avec l'erreur `could not translate host name «dpg-xxxx-a» to address`. En creusant dans la documentation Render, j'ai compris que la base PostgreSQL du plan gratuit est automatiquement supprimée après une durée d'inactivité prolongée : l'hôte référencé dans `DATABASE_URL` n'existait tout simplement plus. J'ai recréé une base, mis à jour la variable `DATABASE_URL` sur Render, puis redéployé — en actant la perte des dinosaures ajoutés en production (les images, elles, étaient préservées séparément sur Cloudinary, ce qui a confirmé l'intérêt d'avoir externalisé le stockage des médias).
- Secrets exposés — `SECRET_KEY` en clair dans le code et `DEBUG=True` en production (voir le détail du risque et la correction en section 3.7). Au-delà du correctif technique (sortie en variables d'environnement), cet incident a changé ma façon d'aborder le déploiement : je vérifie désormais systématiquement la valeur de `DEBUG` avant chaque mise en production, plutôt que de m'appuyer sur la valeur par défaut du code.
- Dépendance manquante — le déploiement échouait dès l'installation des dépendances sur Render, alors que l'application fonctionnait sans problème en local. En comparant `requirements.txt` entre les deux environnements, j'ai trouvé deux causes cumulées : `python-dotenv` était utilisé dans le code (`import dotenv`) mais absent du fichier, et ce même fichier avait été accidentellement sauvegardé en encodage UTF-16 par mon éditeur plutôt qu'en UTF-8, ce que `pip` ne sait pas lire correctement. J'ai régénéré `requirements.txt` proprement avec `pip freeze` et vérifié son encodage avant de le recommiter.
- Timeout Gunicorn — l'upload d'une image de 8,56 Mo pour une fiche dinosaure provoquait une erreur de timeout côté serveur, le worker Gunicorn étant tué avant la fin du transfert vers Cloudinary. Le timeout par défaut de Gunicorn (30 secondes) suffit pour des requêtes classiques mais pas pour un envoi de fichier lourd combiné à la latence réseau vers un service tiers. J'ai augmenté ce timeout à 120 secondes dans la commande de démarrage (`gunicorn --timeout 120`), tout en notant que la vraie limite à corriger à terme serait une compression de l'image côté client avant l'envoi plutôt que d'allonger indéfiniment le délai serveur.
- Variables Cloudinary mal nommées — `ValueError: Must supply api_key`, alors que les variables Cloudinary étaient bien présentes dans le dashboard Render. En comparant précisément les noms, j'ai découvert qu'elles avaient été saisies telles quelles depuis les intitulés affichés sur le tableau de bord Cloudinary (`api_key`, `api_secret`, `cloud_name`), alors que mon code, via `os.environ.get()`, attend une correspondance exacte et sensible à la casse avec `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET` et `CLOUDINARY_CLOUD_NAME`. Un simple rapprochement visuel entre les deux tableaux de variables a suffi à repérer l'écart et à le corriger.

Cette expérience de déploiement a été la plus formatrice du projet : elle m'a confronté à des problèmes concrets de configuration dépendante de l'environnement, que je n'avais jamais rencontrés en développement local.

Environment Variables

Export

Edit

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#)

KEY	VALUE		
CLOUDINARY_API_KEY	*****		
CLOUDINARY_API_SECRET	*****		
CLOUDINARY_CLOUD_NAME	*****		
DATABASE_URL	*****		

```
Sep 17
11:45:18 AM [5xw4h] File "/opt/render/project/src/.venv/lib/python3.13/site-packages/django/db/backends/base/base.py", line 328, in cursor
11:45:18 AM [5xw4h]     return self._cursor()
11:45:18 AM [5xw4h]     ^^^^^^^^^^^^^
11:45:18 AM [5xw4h] File "/opt/render/project/src/.venv/lib/python3.13/site-packages/django/db/backends/base/base.py", line 296, in _cursor
11:45:18 AM [5xw4h]     self.ensure_connection()
11:45:18 AM [5xw4h]     ^^^^^^^^^^^^^
11:45:18 AM [5xw4h] File "/opt/render/project/src/.venv/lib/python3.13/site-packages/django/utils/asyncio.py", line 26, in inner
11:45:18 AM [5xw4h]     return func(*args, **kwargs)
11:45:18 AM [5xw4h] File "/opt/render/project/src/.venv/lib/python3.13/site-packages/django/db/backends/base/base.py", line 278, in ensure_connection
11:45:18 AM [5xw4h]     with self.wrap_database_errors:
11:45:18 AM [5xw4h]         ^^^^^^^^^^^^^^^^^^^^^^^^^
11:45:18 AM [5xw4h] File "/opt/render/project/src/.venv/lib/python3.13/site-packages/django/db/utils.py", line 91, in __exit__
11:45:18 AM [5xw4h]     raise dj_exc_value.with_traceback(traceback) from exc_value
11:45:18 AM [5xw4h] File "/opt/render/project/src/.venv/lib/python3.13/site-packages/django/db/backends/base/base.py", line 279, in ensure_connection
11:45:18 AM [5xw4h]     self.connect()
11:45:18 AM [5xw4h]     ^^^^^^^^^^^^^
11:45:18 AM [5xw4h] File "/opt/render/project/src/.venv/lib/python3.13/site-packages/django/db/utils/asyncio.py", line 26, in inner
11:45:18 AM [5xw4h]     return func(*args, **kwargs)
11:45:18 AM [5xw4h] File "/opt/render/project/src/.venv/lib/python3.13/site-packages/django/db/backends/base/base.py", line 256, in connect
11:45:18 AM [5xw4h]     self.connection = self.get_new_connection(conn_params)
11:45:18 AM [5xw4h]     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
11:45:18 AM [5xw4h] File "/opt/render/project/src/.venv/lib/python3.13/site-packages/django/db/utils/asyncio.py", line 26, in inner
11:45:18 AM [5xw4h]     return func(*args, **kwargs)
11:45:18 AM [5xw4h] File "/opt/render/project/src/.venv/lib/python3.13/site-packages/django/db/backends/postgresql/base.py", line 359, in get_new_connection
11:45:18 AM [5xw4h]     connection = self.Database.connect(**conn_params)
11:45:18 AM [5xw4h]     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
11:45:18 AM [5xw4h] File "/opt/render/project/src/.venv/lib/python3.13/site-packages/django/db/backends/postgresql/base.py", line 122, in connect
11:45:18 AM [5xw4h]     conn = _connect(dsn, connection_factory=connection_factory, **kwargs)
11:45:18 AM [5xw4h] django.db.utils.OperationalError: could not translate host name "dog-g9ti2hw4l7fo73eflrkg-a" to address: Name or service not known
11:45:18 AM [5xw4h]
11:45:19 AM [5xw4h] ==> No open ports detected, continuing to scan...
11:45:19 AM [5xw4h] ==> Docs on specifying a port: https://render.com/docs/web-services#port-binding
```

4. Tests et validation

4.1 Méthodologie de test

La fonctionnalité la plus représentative du projet, `verifier_compatibilite()`, est couverte par une suite de tests automatisés écrits avec le framework `TestCase` de Django (`jurassic_site/tests.py`), exécutée avec la commande `python manage.py test jurassic_site`.

`TestCase` — et pas une simple classe Python — est un choix déterminant : Django crée une base de données de test à part, exécute chaque test dedans, puis annule tout à la fin (rollback). Résultat : les tests ne touchent jamais la vraie base de données, et un test ne peut pas polluer le suivant. Chaque test suit le même schéma en trois temps : Arrange (préparation des données, ici la création de vrais objets `Dinosaure` en base de test, nécessaire car `verifier_compatibilite()` appelle des méthodes comme `dino.naime_pas.all()` qui ont besoin d'un objet réellement enregistré), Act (appel de la fonction testée), Assert (vérification du résultat obtenu contre le résultat attendu).

Chaque règle de compatibilité identifiée en section 3.3 a son propre test, y compris dans les deux sens d'appel lorsque la règle repose sur une relation asymétrique (``aime``, ``naime_pas``, ``naime_pas_types``, ``naime_pas_familles``) — un point important puisque `verifier_compatibilite(dino1, dino2)` n'est pas censé donner un résultat différent de `verifier_compatibilite(dino2, dino1)`, et seul un test dans les deux sens permet de le garantir.

Exemple d'un test complet

Pour illustrer le schéma Arrange / Act / Assert décrit ci-dessus, voici le test le plus simple de la suite dans son intégralité :

```
def test_carnivore_et_herbivore_incompatibles(self):
    # ARRANGE : préparation des données
    rex = Dinosaure.objects.create(nom="T-Rex", type="Carnivore")
    tricera = Dinosaure.objects.create(nom="Tricératops", type="Herbivore")

    # ACT : exécution de la fonction testée
    compatible, raison = verifier_compatibilite(rex, tricera)

    # ASSERT : vérification du résultat
    self.assertFalse(compatible)
    self.assertIn("carnivores et herbivores", raison)
```

Ce test vérifie à la fois la valeur booléenne retournée (compatible doit être False) et le contenu du message explicatif (raison doit mentionner la règle appliquée), ce qui permet de détecter non seulement une régression sur le résultat mais aussi un message d'erreur devenu incohérent avec la règle réellement appliquée.

Les 14 tests sont volontairement indépendants les uns des autres : chacun crée ses propres objets `Dinosaure` en base de test plutôt que de réutiliser un jeu de données commun, ce qui alourdit légèrement l'écriture de chaque test mais garantit qu'un test ne peut jamais échouer à cause d'un état laissé par un

autre — un principe qui s'est avéré utile lors de l'ajout tardif du test #14, qui n'a nécessité aucune adaptation des 13 tests existants.

4.2 Jeu d'essai

Résultat de l'exécution de la suite complète : 14/14 tests réussis.

#	Cas testé	Données en entrée	Résultat
1	Carnivore vs Herbivore	T-Rex (Carnivore), Tricératops (Herbivore)	Incompatible — conforme
2	Herbivore vs Herbivore	Tricératops, Edmontosaure (Herbivore)	Compatible — conforme
3	Reptile marin vs terrestre	Mosasaure (Reptile marin), Tricératops (Herbivore)	Incompatible — conforme
4	Reptile marin vs Reptile marin	Mosasaure, Plésiosaure (Reptile marin)	Compatible — conforme
5	Incompatibilité individuelle (naime_pas)	Pentaceratops, Nasutocératops (naime_pas)	Incompatible — conforme
6	Incompatibilité de famille (2 sens)	Ankylosaure (naime_pas_familles=Sauropodes), Diplodocus (Sauropodes)	Incompatible dans les 2 sens — conforme
7	Reptile volant vs Reptile volant	Dimorphodon, Ptéranodon (Reptile volant)	Compatible — conforme
8	Reptile volant vs terrestre	Dimorphodon, Tricératops (Herbivore)	Incompatible — conforme
9	Reptile volant vs Reptile marin	Dimorphodon, Mosasaure	Incompatible (règle marine prioritaire) — conforme
10	Piscivore vs Herbivore	Spinosaure (Piscivore), Tricératops (Herbivore)	Incompatible — conforme
11	Piscivore vs Carnivore	Spinosaure, Velociraptor (Carnivore)	Incompatible — conforme
12	Affinité positive (aime, 2 sens)	Diplodocus (aime Gallimimus), Gallimimus	Compatible dans les 2 sens — conforme
13	Incompatibilité de type (naime_pas_types, 2 sens)	T-Rex (naime_pas_types=Carnivore), Velociraptor (Carnivore)	Incompatible dans les 2 sens — conforme
14	Exception charognard (2 sens)	Compsognathus (Carnivore, famille=Charognard), Tricératops (Herbivore)	Compatible dans les 2 sens malgré la règle carnivore/herbivore — conforme

Le test #14 (exception charognard) a été ajouté en dernier, après la relecture du code faite pour ce dossier (section 3.3) : il n'existait pas dans la première version de la suite, et son ajout illustre bien l'intérêt des tests comme filet de sécurité — s'assurer qu'une règle spéciale découverte a posteriori reste correcte dans le temps, y compris si le code environnant est modifié plus tard.

```
(venv) PS E:\Django\jurassic_site> python manage.py test jurassic_site
Found 14 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 14 tests in 0.024s

OK
Destroying test database for alias 'default'...
(venv) PS E:\Django\jurassic_site>
```

Cette suite ne couvre que la fonction `verifier_compatibilite()`, qui concentre l'essentiel de la logique métier du projet. Les vues elles-mêmes (ajout, modification, suppression de fiches, API `get_compatibles`) et les formulaires ne sont pas couverts par des tests automatisés : c'est une limite assumée de la suite actuelle, à combler en priorité si le projet devait évoluer, en particulier sur `get_compatibles()` qui agrège plusieurs appels à `verifier_compatibilite()` et pourrait cacher un bug d'agrégation qu'un test unitaire sur la seule fonction de compatibilité ne détecterait pas.

Matrice de traçabilité exigences / tests

Pour clore la partie validation, ce tableau relie chaque exigence fonctionnelle du cahier des charges (section 1.3) aux tests qui la couvrent effectivement :

Exigence	Test(s) associé(s)	Statut
EF1 — Compatibilité multi-sélection mise à jour sans rechargement	Tests #1 à #13 (règles individuelles) + revue manuelle de l'intégration AJAX (section 3.4)	Couvert
EF2 — Filtrage par type, régime, recherche par nom	Non couvert par un test automatisé (JavaScript côté client, hors périmètre des tests Django)	Non couvert
EF3 — CRUD complet sur les fiches dinosaures	Non couvert par un test automatisé (formulaires et vues, voir limite notée en section 3.4)	Non couvert
EF4 — Cohérence des règles de compatibilité dans les deux sens	Tests #5, #6, #12, #13, #14	Couvert
EF5 — Persistance des médias après redéploiement	Validé empiriquement lors de l'incident de déploiement n°1 (section 3.8), non automatisé	Validé manuellement

Ce tableau rend explicite ce qui n'était qu'implicite dans le reste du dossier : sur cinq exigences fonctionnelles, deux (EF2 et EF3) ne sont couvertes par aucun test automatisé, ce qui rejoint directement la limite notée en section 3.4 et confirme que la priorité d'évolution la plus utile ne serait pas d'ajouter des fonctionnalités, mais de sécuriser celles qui existent déjà.

5. Documentation

5.1 Documentation utilisateur

L'application propose quatre pages principales, accessibles depuis le menu de navigation :

- Accueil : page de présentation du site, point d'entrée vers les trois autres pages via le menu de navigation.
- Liste des dinosaures : affiche l'ensemble des dinosaures enregistrés sous forme de fiches, avec un filtre par type (Herbivore, Carnivore, Piscivore, Reptile marin, Reptile volant) permettant de restreindre

l'affichage. Chaque fiche donne accès à la modification ou la suppression du dinosaure correspondant, sans quitter le contexte de la liste.

- Ajouter un dinosaure : formulaire de création d'une fiche (nom, type, famille, régime alimentaire, photo). Les affinités et incompatibilités ne sont volontairement pas proposées à la création : elles ne prennent sens qu'une fois plusieurs dinosaures déjà enregistrés, elles sont donc réservées au formulaire de modification (page suivante).
- Simuler un enclos : l'utilisateur sélectionne un ou plusieurs dinosaures dans une liste avec recherche intégrée (django-select2). À chaque sélection, la liste des dinosaures compatibles avec l'ensemble de l'enclos se met à jour automatiquement via l'API décrite en section 3.4, avec le motif de compatibilité ou d'incompatibilité affiché pour chaque espèce — c'est la page qui concentre l'essentiel de la valeur ajoutée du projet par rapport à une simple base de données de dinosaures.





Liste des dinosaures

Filtrer par type :

Tous



acrocanthosaurus

Type : Carnivore
Nourriture : Viandes, Proie

Modifier

Supprimer



albertosaure

Type : Carnivore
Nourriture : Proie

Modifier

Supprimer



allosaure

Type : Carnivore
Nourriture : Viandes, Proie

Modifier

Supprimer



amargasaurus

Type : Herbivore
Nourriture : Fibre au ras du sol

Modifier

Supprimer



Liste des dinosaures

Filtrer par type :

Reptile marin



attenborosaurus

Type : Reptile marin
Nourriture : Poisson

Modifier

Supprimer



ichtyosaure

Type : Reptile marin
Nourriture : Poisson

Modifier

Supprimer



liopleurodon

Type : Reptile marin
Nourriture : Poisson

Modifier

Supprimer



mosasaure

Type : Reptile marin
Nourriture : Poisson, Requin

Modifier

Supprimer



Ajouter un dinosaure

Nom:

Type:

Famille:

Nourriture:

- ☐ Viandes
- ☐ Proie
- ☐ Poisson
- ☐ Requin
- ☐ Fibre au ras du sol
- ☐ Fruit au ras du sol
- ☐ Fibre en hauteur
- ☐ Fruit en hauteur
- ☐ Feuille au ras du sol
- ☐ Feuille en hauteur
- ☐ Noix au ras du sol
- ☐ Noix en hauteur

Image:

© 2025 Jurassic Site

attenborosaurus

Nom:

Type:

Famille:

Nourriture:

☒ Poisson

Image:

Clear

Change:

Aime:

N'aime pas:

Types incompatibles:

Familles incompatibles:

5.2 Documentation technique

Structure du projet : l'application est regroupée dans une unique app Django, `jurassic\_site`, qui porte les modèles, vues, formulaires, templates et tests. Les médias utilisateurs (photos de dinosaures) sont stockés sur Cloudinary ; les fichiers statiques (CSS, images fixes) sont servis via WhiteNoise, y compris en production.

Routes exposées par l'application (`jurassic\_site/urls.py`) :

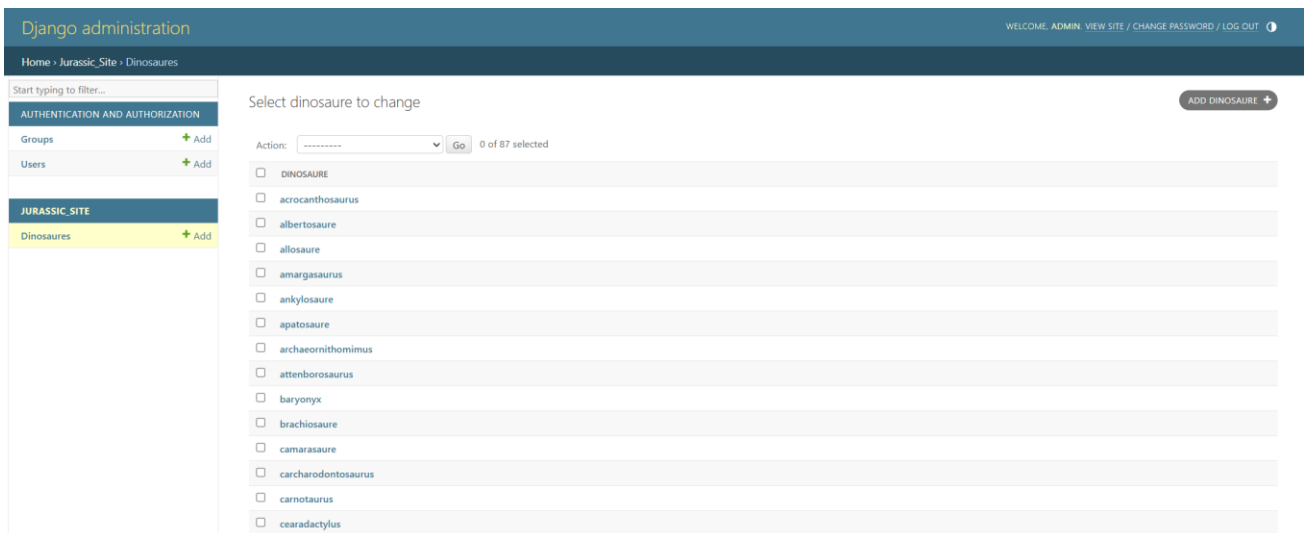
Route	Vue	Rôle
/	home	Page d'accueil
/liste/	liste_dinos	Liste des dinosaures, avec filtre par type
/ajouter/	ajouter_dino	Formulaire de création d'une fiche dinosaure
/modifier/<id>/	modifier_dino	Formulaire de modification (affinités, incompatibilités, image)
/supprimer/<id>/	supprimer_dino	Confirmation puis suppression d'une fiche
/enclos/	enclos	Simulateur d'enclos (sélection multiple + recherche)
/api/compatibles/<ids>/	get_compatibles	API JSON consommée en AJAX par le simulateur
/admin/	(Django admin)	Interface d'administration standard Django

Variables d'environnement requises (fichier `.env` en local, variables d'environnement du service en production) : ``SECRET_KEY``, ``DEBUG``, ``CLOUDINARY_CLOUD_NAME``, ``CLOUDINARY_API_KEY``, ``CLOUDINARY_API_SECRET``, ``DATABASE_URL`` (production uniquement, absente en local où SQLite est utilisé par défaut).

Pour la maintenance future, deux points de vigilance identifiés en écrivant ce dossier (section 3.7) : l'absence d'authentification sur les routes de modification/suppression, et la nécessité de garder ``DEBUG=False`` en production à chaque redéploiement — un oubli sur ce dernier point a déjà causé l'un des cinq incidents documentés en section 3.8.

Interface d'administration

L'application s'appuie également sur l'interface d'administration intégrée à Django (accessible sur /admin/), activée par défaut via 'django.contrib.admin' dans INSTALLED_APPS. Elle donne un accès direct aux tables Dinosaur et Nourriture sans avoir à écrire de vue ni de template dédiés, ce qui en a fait un outil pratique pour vérifier ou corriger rapidement des données pendant le développement (par exemple relire les incompatibilités enregistrées sur une fiche), en complément des pages publiques de l'application.



Procédure de déploiement sur Render

Étapes suivies pour la mise en production, telles qu'elles ont été stabilisées après les cinq incidents de la section 3.8 :

- Créer le service web sur Render, connecté au dépôt GitHub Ivann212/JWE (branche main).
- Créer une base PostgreSQL managée sur Render, et copier son DATABASE_URL interne dans les variables d'environnement du service.
- Renseigner les variables d'environnement du service : SECRET_KEY, DEBUG=False, CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET, DATABASE_URL — en respectant strictement la casse attendue par os.environ.get() (voir section 3.8).
- Définir la commande de build (pip install -r requirements.txt suivi de python manage.py collectstatic --noinput et python manage.py migrate) et la commande de démarrage (gunicorn jurassic_site.wsgi --timeout 120, voir section 3.8 pour la justification du timeout).
- Déclencher un premier déploiement manuel, vérifier les logs Render en cas d'échec plutôt que de redéployer à l'aveugle (c'est ainsi que les incidents 1, 3 et 5 ont été diagnostiqués).
- Une fois stable, laisser le déploiement automatique sur chaque push vers main.

6. Bilan personnel

JWE (Jurassic World Enclos) est une application web développée en Django, qui calcule la compatibilité entre espèces de dinosaures du jeu vidéo Jurassic World Evolution 2, dans le but d'aider à composer des enclos optimisés sans avoir à consulter manuellement chaque fiche en jeu. Développée seule, initialement en local avec SQLite, elle a été déployée en ligne sur Render avec PostgreSQL et Cloudinary pour le stockage des médias. Le projet est couvert par une suite de 14 tests unitaires validant l'ensemble des règles de compatibilité.

Ce projet m'a permis de développer une réelle compétence en rédaction de tests unitaires. Avant ce projet, je n'en voyais pas l'intérêt, au point d'accepter de perdre des points sur des évaluations plutôt que d'en écrire. Le fait de travailler sur une application concrète, construite autour d'un sujet qui me passionne, m'a permis de comprendre pour la première fois à quoi servaient réellement les tests : non pas prouver quelque chose que je sais déjà, mais me protéger contre des régressions futures et documenter le comportement attendu de mon code.

Mes points forts sur ce projet ont été ma capacité à rédiger au brouillon la logique de ma fonction de compatibilité avant de la coder, ainsi que ma capacité à remettre en question a posteriori des choix techniques que je n'avais pas moi-même pensés initialement — par exemple en questionnant la pertinence du choix ``symmetrical=False`` sur mes relations de modèle, et en comprenant ses implications concrètes sur le comportement de l'application.

Mes points faibles ont porté sur la mise en place technique du projet, notamment l'intégration d'un service externe comme Cloudinary, ainsi que sur la syntaxe Python elle-même : j'arrivais à structurer logiquement ce que je voulais faire, mais je manquais parfois du vocabulaire technique précis pour le traduire en code fonctionnel. C'est d'ailleurs l'axe de progression que je retiens comme prioritaire pour la suite : renforcer ma maîtrise des règles de base du langage Python (comportement des structures de contrôle, syntaxe des fonctions), pour être capable de proposer moi-même des façons alternatives d'écrire une même logique — un point sur lequel je m'attends à être interrogé par le jury.

Ce projet m'a aussi confronté, plus que mes précédents travaux scolaires, à la nécessité de documenter mes propres décisions au moment où je les prends plutôt qu'après coup : plusieurs points relevés dans ce dossier (la règle des charognards en section 3.3, la limite de performance en section 3.4, ou encore le choix du stockage en chaîne de caractères pour les incompatibilités de catégorie en section 2.2) ne figuraient dans aucune note écrite avant que je ne les rédige pour ce dossier, alors qu'il s'agit de décisions ou de constats faits en cours de développement. À l'avenir, je compte tenir un journal de bord plus systématique au fil du projet plutôt que de reconstituer ce raisonnement a posteriori, ce qui aurait aussi facilité la rédaction de ce document.

Sur le plan méthodologique, ce projet a aussi changé ma façon d'aborder une session de développement : plutôt que de coder une fonctionnalité de bout en bout puis de revenir corriger les problèmes constatés, j'ai pris l'habitude, en particulier pour `verifier_compatibilite()`, d'écrire d'abord la liste des cas à couvrir avant d'écrire le code — une discipline directement issue de la rédaction des tests de la section 4.2, qui m'a servi ensuite pour la logique de filtrage côté client (section 3.6) et pour la vérification des variables d'environnement au moment du déploiement (section 3.8).

Perspectives d'évolution

Trois évolutions se dégagent naturellement du bilan de ce projet, par ordre de priorité si je devais continuer à y travailler après l'obtention du titre.

- Court terme : intégrer la maquette de refonte (section 2.4) à l'application Django en production, en remplaçant le modèle 3D générique par un modèle dédié aux dinosaures du site, et couvrir les vues actuellement non testées (section 4.2) par de nouveaux tests.
- Moyen terme : ajouter un système d'authentification pour restreindre les actions d'ajout/modification/suppression aux utilisateurs identifiés (section 3.7), ce qui ouvrirait la voie à un usage multi-utilisateur si le projet devait dépasser mon usage personnel.
- Long terme : explorer une API publique ou un jeu de données partagé pour remplacer la saisie manuelle des fiches dinosaures (section 1.2), qui reste la principale contrainte opérationnelle du projet aujourd'hui.

Annexes

Code source complet des fichiers cités dans ce dossier, pour référence.

Annexe A — `models.py`

```
from django.db import models
from cloudinary.models import CloudinaryField

class Nourriture(models.Model):
    nom = models.CharField(max_length=100, unique=True)

    def __str__(self):
        return self.nom
```

```

class Meta:
    verbose_name_plural = "Nourritures"

class Dinsaure(models.Model):
    TYPE_CHOICES = [
        ('Herbivore', 'Herbivore'),
        ('Carnivore', 'Carnivore'),
        ('Piscivore', 'Piscivore'),
        ('Reptile marin', 'Reptile marin'),
        ('Reptile volant', 'Reptile volant'),
    ]

    FAMILLE_CHOICES = [
        ('Sauropodes', 'Sauropodes'),
        ('Pachycéphalosauridé', 'Pachycéphalosauridé'),
        ('Hadrosauridé', 'Hadrosauridé'),
        ('Ankylosauridé', 'Ankylosauridé'),
        ('Cératopsien', 'Cératopsien'),
        ('Ornithomimosaurien', 'Ornithomimosaurien'),
        ('Charognard', 'Charognard'),
        ('Hybrides', 'Hybrides'),
    ]

    # Informations de base
    nom = models.CharField(max_length=100)
    type = models.CharField(max_length=50, choices=TYPE_CHOICES)
    famille = models.CharField(max_length=100, choices=FAMILLE_CHOICES, blank=True,
null=True) # ☒ Optionnel
    nourriture = models.ManyToManyField(Nourriture, blank=True)
    image = CloudinaryField('image', blank=True, null=True)

    # Compatibilités individuelles
    aime = models.ManyToManyField('self', symmetrical=False, blank=True,
related_name='aimé_par')
    naim_pas = models.ManyToManyField('self', symmetrical=False, blank=True,
related_name='détesté_par')

    # Incompatibilités par catégories
    naim_pas_types = models.CharField(
        max_length=200,
        blank=True,
        help_text="Types incompatibles (séparés par des virgules)"
    )
    naim_pas_familles = models.CharField(
        max_length=200,
        blank=True,
        help_text="Familles incompatibles (séparées par des virgules)"
    )

    def __str__(self):
        return self.nom

```

```

class Meta:
    verbose_name_plural = "Dinosaures"
    ordering = ['nom']

```

Annexe B — forms.py

```

from django import forms
from .models import Dinsaure, Nourriture
from django_select2.forms import Select2MultipleWidget
class DinsaureAddForm(forms.ModelForm):
    class Meta:
        model = Dinsaure
        fields = ['nom', 'type', 'famille', 'nourriture', 'image']
        widgets = {
            'nourriture': forms.CheckboxSelectMultiple(attrs={'size': '3'})
        }

class DinsaureEditForm(forms.ModelForm):
    # Champs pour la sélection multiple
    naime_pas_types_select = forms.MultipleChoiceField(
        choices=Dinsaure.TYPE_CHOICES,
        required=False,
        widget=forms.SelectMultiple(attrs={
            'class': 'select2-multiple',
            'data-placeholder': 'Sélectionner des types incompatibles...'
        }),
        label="Types incompatibles"
    )

    naime_pas_familles_select = forms.MultipleChoiceField(
        choices=Dinsaure.FAMILLE_CHOICES,
        required=False,
        widget=forms.SelectMultiple(attrs={
            'class': 'select2-multiple',
            'data-placeholder': 'Sélectionner des familles incompatibles...'
        }),
        label="Familles incompatibles"
    )

    class Meta:
        model = Dinsaure
        fields = ['nom', 'type', 'famille', 'nourriture', 'image',
            'aime', 'naime_pas']
        widgets = {
            'aime': forms.SelectMultiple(attrs={
                'class': 'select2-multiple',
                'data-placeholder': 'Dinosaures aimés...'
            }),
            'naime_pas': forms.SelectMultiple(attrs={

```

```

        'class': 'select2-multiple',
        'data-placeholder': 'Dinosaures détestés...'
    )),
    'nourriture': forms.SelectMultiple(attrs={
        'class': 'select2-multiple',
        'data-placeholder': 'Sélectionner la nourriture...'
    )),
}

def __init__(self, *args, **kwargs):
    super().__init__(*args, **kwargs)
    if self.instance.pk:
        self.fields['aime'].queryset = Dinosauré.objects.exclude(pk=self.instance.pk)
        self.fields['naime_pas'].queryset =
Dinosauré.objects.exclude(pk=self.instance.pk)

        # Pré-remplir les incompatibilités
        if self.instance.naime_pas_types:
            self.fields['naime_pas_types_select'].initial =
self.instance.naime_pas_types.split(',')
        if self.instance.naime_pas_familles:
            self.fields['naime_pas_familles_select'].initial =
self.instance.naime_pas_familles.split(',')

    def save(self, commit=True):
        instance = super().save(commit=False)

        # Sauvegarder les incompatibilités
        instance.naime_pas_types = ','.join(self.cleaned_data['naime_pas_types_select'])
        instance.naime_pas_familles =
','.join(self.cleaned_data['naime_pas_familles_select'])

        if commit:
            instance.save()
            self.save_m2m()

        return instance

```

Annexe C — views.py

```

from django.shortcuts import render, redirect, get_object_or_404
from .forms import DinosauréAddForm, DinosauréEditForm
from .models import Dinosauré
from django.http import HttpResponseRedirect
from django.urls import reverse
from django.http import JsonResponse
import cloudinary.uploader

# Page d'accueil (liste des dinos)
def home(request):
    return render(request, 'home.html')

```

```

def liste(request):
    dinos = Dinsaure.objects.all()
    return render(request, 'dinos_list.html', {'dinos': dinos})
# Ajouter un dinosaure
def ajouter_dino(request):
    if request.method == 'POST':
        form = DinsaureAddForm(request.POST, request.FILES)
        if form.is_valid():
            dino = form.save(commit=False) # On crée le dino sans sauvegarder encore les
M2M
            dino.save() # On sauvegarde l'objet principal
            form.save_m2m() # Maintenant on peut sauvegarder les champs ManyToMany
            return redirect('home')
        else:
            form = DinsaureAddForm()
            return render(request, 'ajouter_dino.html', {'form': form})

# Modifier un dinosaure pour ajouter "aime" et "n'aime pas"
def modifier_dino(request, dino_id):
    dino = get_object_or_404(Dinsaure, id=dino_id)
    if request.method == 'POST':
        form = DinsaureEditForm(request.POST, request.FILES, instance=dino)
        oldImage = dino.image
        if form.is_valid():
            if 'image' in request.FILES and oldImage:
                cloundinary.uploader.destroy(oldImage.public_id)
            form.save()
            return redirect('liste_dinos')

        else:
            form = DinsaureEditForm(instance=dino)
            return render(request, 'formulaire_dino.html', {'form': form, 'dino': dino})

def liste_dinos(request):
    print("View liste_dinos appelée")

    type_filtre = request.GET.get('type', '') # Récupère le filtre depuis l'URL

    # Si un type est choisi, on filtre
    if type_filtre and type_filtre != 'Tous':
        dinos = Dinsaure.objects.filter(type=type_filtre)

    else:
        dinos = Dinsaure.objects.all()

    # On récupère la liste des types possibles
    types_disponibles = [choice[0] for choice in Dinsaure.TYPE_CHOICES]
    types_disponibles.insert(0, "Tous")

    return render(request, 'dinos_list.html', {

```

```

        'dinos': dinos,
        'type_filtre': type_filtre,
        'types_disponibles': types_disponibles
    })
def supprimer_dino(request, dino_id):
    dino = get_object_or_404(Dinosaure, id=dino_id)
    if request.method == "POST":
        dino.delete()
        return redirect('liste_dinos')
    return render(request, 'supprimer_dino.html', {'dino': dino})

def enclos(request):
    dinos = Dinosaure.objects.all()
    return render(request, 'enclos.html', {'dinos': dinos})

def enclos(request):
    """Page principale de l'enclos"""
    dinosaures = Dinosaure.objects.all().order_by('nom')
    return render(request, 'enclos.html', {'dinosaures': dinosaures})

def verifier_compatibilite(dino1, dino2):
    """
    Vérifie si deux dinosaures sont compatibles
    Retourne (compatible: bool, raison: str)
    """

    # Règles de base par type (environnement)
    types_aquatiques = ['Reptile marin']
    types_aeriens = ['Reptile volant']
    types_terrestres = ['Carnivore', 'Herbivore', 'Piscivore']

    # 1. Les reptiles marins sont uniquement compatibles entre eux
    if dino1.type in types_aquatiques or dino2.type in types_aquatiques:
        if dino1.type == dino2.type == 'Reptile marin':
            # Vérifier les autres règles pour les reptiles marins entre eux
            pass
        else:
            return False, "Les reptiles marins nécessitent un enclos aquatique séparé"

    # 2. Les reptiles volants sont uniquement compatibles entre eux
    if dino1.type in types_aeriens or dino2.type in types_aeriens:
        if dino1.type == dino2.type == 'Reptile volant':
            # Vérifier les autres règles pour les reptiles volants entre eux
            pass
        else:
            return False, "Les reptiles volants nécessitent une volière séparée"

    # 3bis. Les charognards s'entendent avec toutes les espèces terrestres :

```

```

# on ignore les règles de type (carnivore/herbivore/piscivore) ci-dessous
# si l'un des deux dinosaures appartient à la famille des charognards.
est_charognard = dino1.famille == 'Charognard' or dino2.famille == 'Charognard'

if not est_charognard:
    # 3. Les carnivores ne sont pas compatibles avec les herbivores
    if (dino1.type == 'Carnivore' and dino2.type == 'Herbivore') or \
        (dino1.type == 'Herbivore' and dino2.type == 'Carnivore'):
        return False, "Les carnivores et herbivores ne peuvent pas cohabiter"

    # 4. Les piscivores ne sont compatibles ni avec carnivores ni avec herbivores
    if dino1.type == 'Piscivore' and dino2.type in ['Carnivore', 'Herbivore']:
        return False, "Les piscivores ne sont pas compatibles avec ce type"
    if dino2.type == 'Piscivore' and dino1.type in ['Carnivore', 'Herbivore']:
        return False, "Les piscivores ne sont pas compatibles avec ce type"

# 5. Vérifier les incompatibilités individuelles
if dino2 in dino1.naime_pas.all():
    return False, f"{dino1.nom} n'aime pas {dino2.nom}"
if dino1 in dino2.naime_pas.all():
    return False, f"{dino2.nom} n'aime pas {dino1.nom}"

# 6. Vérifier les incompatibilités de type
if dino1.naime_pas_types:
    types_interdits = [t.strip() for t in dino1.naime_pas_types.split(',') if
t.strip()]
    if dino2.type in types_interdits:
        return False, f"{dino1.nom} n'aime pas les {dino2.type}s"

    if dino2.naime_pas_types:
        types_interdits = [t.strip() for t in dino2.naime_pas_types.split(',') if
t.strip()]
        if dino1.type in types_interdits:
            return False, f"{dino2.nom} n'aime pas les {dino1.type}s"

# 7. Vérifier les incompatibilités de famille
if dino1.naime_pas_familles and dino2.famille:
    familles_interdites = [f.strip() for f in dino1.naime_pas_familles.split(',') if
f.strip()]
    if dino2.famille in familles_interdites:
        return False, f"{dino1.nom} n'aime pas les {dino2.famille}"

    if dino2.naime_pas_familles and dino1.famille:
        familles_interdites = [f.strip() for f in dino2.naime_pas_familles.split(',') if
f.strip()]
        if dino1.famille in familles_interdites:
            return False, f"{dino2.nom} n'aime pas les {dino1.famille}"

# 8. Vérifier les compatibilités positives (bonus si défini)
if dino1.aime.filter(id=dino2.id).exists():
    return True, f"{dino1.nom} apprécie {dino2.nom} !"
if dino2.aime.filter(id=dino1.id).exists():
    return True, f"{dino2.nom} apprécie {dino1.nom} !"

```

```

# Si aucune incompatibilité trouvée
return True, "Compatible"

def get_compatibles(request, dino_ids):
    """
    API pour obtenir les dinosaures compatibles avec un ou plusieurs dinosaures
    sélectionnés
    dino_ids: chaîne d'IDs séparés par des virgules (ex: "1,3,5")
    """
    try:
        ids_list = [int(id.strip()) for id in dino_ids.split(',')]
        dinos_selectionnees = Dinosaur.objects.filter(id__in=ids_list)

        if not dinos_selectionnees.exists():
            return JsonResponse({'success': False, 'error': 'Aucun dinosaure trouvé'},
                                status=404)

        # Exclure les dinosaures déjà sélectionnés
        tous_les_dinos = Dinosaur.objects.exclude(id__in=ids_list)

        compatibles = []

        for dino in tous_les_dinos:
            # Vérifier la compatibilité avec TOUS les dinosaures de l'enclos
            est_compatible_avec_tous = True
            raisons = []

            for dino_enclos in dinos_selectionnees:
                est_compatible, raison = verifier_compatibilite(dino_enclos, dino)

                if not est_compatible:
                    est_compatible_avec_tous = False
                    raisons.append(raison)
                    break # Pas besoin de vérifier les autres si déjà incompatible
            else:
                if "apprécie" in raison: # Relation positive
                    raisons.append(raison)

        # Ne garder que les dinosaures compatibles avec TOUS
        if est_compatible_avec_tous:
            compatibles.append({
                'id': dino.id,
                'nom': dino.nom,
                'type': dino.type,
                'famille': dino.famille or 'Non définie',
                'image': dino.image.url if dino.image else None,
                'raison': raisons[0] if raisons else 'Compatible'
            })

        return JsonResponse({
            'success': True,

```

```

        'dinos_selectionnes': [{
            'id': d.id,
            'nom': d.nom,
            'type': d.type,
            'famille': d.famille or 'Non définie',
            'image': d.image.url if d.image else None,
        } for d in dinos_selectionnes],
        'compatibles': compatibles,
        'count': len(compatibles)
    })

except Exception as e:
    print(e)
    return JsonResponse({'success': False, 'error': "Une erreur est survenue"},
status=500)

```

Annexe D — tests.py

```

"""
Tests pour la logique de compatibilité entre dinosaures (jurassic_site/views.py).

Comment lancer ces tests :
    python manage.py test jurassic_site

Django va automatiquement chercher les classes qui héritent de TestCase
dans ce fichier, et exécuter chaque méthode qui commence par "test_".
"""

from django.test import TestCase
from .models import Dinosaur
from .views import verifier_compatibilite

class VerifierCompatibiliteTest(TestCase):
    # TestCase (et pas juste une classe Python normale) est important :
    # Django crée une base de données de test à part, exécute chaque
    # test dedans, puis ANNULE tout à la fin (rollback). Résultat :
    # tes tests ne touchent jamais ta vraie base de données, et un test
    # ne peut pas polluer le suivant.

    def test_carnivore_et_herbivore_incompatibles(self):
        # 1) ARRANGE : je prépare mes données.
        # On crée deux VRAIS objets Dinosaur en base de test (nécessaire
        # ici car verifier_compatibilite() appelle des méthodes comme
        # dino.naime_pas.all() ou dino.aime.filter(), qui ont besoin
        # d'un objet réellement sauvegardé pour fonctionner).
        rex = Dinosaur.objects.create(nom="T-Rex", type="Carnivore")
        tricera = Dinosaur.objects.create(nom="Tricératops", type="Herbivore")

        # 2) ACT : j'exécute la fonction que je veux tester.
        compatible, raison = verifier_compatibilite(rex, tricera)

```

```

# 3) ASSERT : je vérifie que le résultat est celui attendu.
# self.assertFalse(x) = "je m'attends à ce que x soit False"
self.assertFalse(compatible)
# self.assertIn(a, b) = "je m'attends à ce que a soit contenu dans b"
self.assertIn("carnivores et herbivores", raison)

# --- À toi de jouer ---
#
# Écris les tests suivants en suivant EXACTEMENT le même schéma
# (Arrange / Act / Assert). Regarde verifier_compatibilite() dans
# views.py pour connaître la règle à vérifier, ça t'aidera à choisir
# les types de dinosaures à créer.
#
def test_deux_herbivores_sont_compatibles(self):
    tricera = Dinosaur.objects.create(nom="Tricératops", type="Herbivore")
    edmont = Dinosaur.objects.create(nom="Edmontosaure", type="Herbivore")

    compatible, raison = verifier_compatibilite(edmont, tricera)

    self.assertTrue(compatible)
    self.assertEqual(raison, "Compatible")
#
def test_reptile_marin_incompatible_avec_terrestre(self):
    mosa = Dinosaur.objects.create(nom="Mosasaure", type="Reptile marin")
    tricera = Dinosaur.objects.create(nom="Tricératops", type="Herbivore")

    compatible, raison = verifier_compatibilite(mosa, tricera)

    self.assertFalse(compatible)
    self.assertIn("enclos aquatique", raison)
#
def test_deux_reptiles_marins_sont_compatibles_entre_eux(self):
    mosa = Dinosaur.objects.create(nom="Mosasaure", type="Reptile marin")
    plesio = Dinosaur.objects.create(nom="Plésiosaure", type="Reptile marin")

    compatible, raison = verifier_compatibilite(mosa, plesio)

    self.assertTrue(compatible)
    self.assertEqual(raison, "Compatible")
#
def test_dino_qui_naime_pas_un_autre_individuellement(self):
    nasu = Dinosaur.objects.create(nom="Nasutocératops", type="Herbivore")
    penta = Dinosaur.objects.create(nom="Pentaceratops", type="Herbivore")
    nasu.naime_pas.add(penta)

    compatible, raison = verifier_compatibilite(penta, nasu)

    self.assertFalse(compatible)
    self.assertIn("n'aime pas", raison)

def test_dino_incompatible_famille(self):
    anky = Dinosaur.objects.create(

```

```

        nom="Ankylosaure", type="Herbivore", famille="Ankylosauridé",
        naime_pas_familles="Sauropodes"
    )
    diplo = Dinsaure.objects.create(
        nom="Diplodocus", type="Herbivore", famille="Sauropodes"
    )

    # Sens 1 : anky.naime_pas_familles contre diplo.famille
    compatible, raison = verifier_compatibilite(anky, diplo)
    self.assertFalse(compatible)
    self.assertIn("n'aime pas les", raison)

    # Sens 2 : même données, ordre inversé → l'autre bloc du code est testé
    compatible2, raison2 = verifier_compatibilite(diplo, anky)
    self.assertFalse(compatible2)
    self.assertIn("n'aime pas les", raison2)
#
def test_deux_reptiles_volants_sont_compatibles_entre_eux(self):
    dimo = Dinsaure.objects.create(nom="Dimorphodon", type="Reptile volant")
    ptera = Dinsaure.objects.create(nom="Ptéranodon", type="Reptile volant")

    compatible, raison = verifier_compatibilite(ptera, dimo)

    self.assertTrue(compatible)
    self.assertEqual(raison, "Compatible")
#
def test_reptile_volant_incompatible_avec_terrestre(self):
    dimo = Dinsaure.objects.create(nom="Dimorphodon", type="Reptile volant")
    tricera = Dinsaure.objects.create(nom="Tricératops", type="Herbivore")

    compatible, raison = verifier_compatibilite(dimo, tricera)

    self.assertFalse(compatible)
    self.assertIn("une volière séparée", raison)
#
def test_reptile_volant_incompatible_avec_marin(self):
    dimo = Dinsaure.objects.create(nom="Dimorphodon", type="Reptile volant")
    mosa = Dinsaure.objects.create(nom="Mosasaure", type="Reptile marin")

    compatible, raison = verifier_compatibilite(dimo, mosa)

    # La règle "reptile marin" est vérifiée avant celle du "reptile
    # volant" dans le code : c'est donc elle qui se déclenche en premier.
    self.assertFalse(compatible)
    self.assertIn("enclos aquatique séparé", raison)
#
def test_piscivore_et_herbivore_incompatibles(self):
    spino = Dinsaure.objects.create(nom="Spinosaure", type="Piscivore")
    tricera = Dinsaure.objects.create(nom="Tricératops", type="Herbivore")

    compatible, raison = verifier_compatibilite(spino, tricera)

    self.assertFalse(compatible)

```

```

        self.assertIn("pas compatibles avec ce type", raison)
#
def test_piscivore_et_carnivore_incompatibles(self):
    spino = Dinsaure.objects.create(nom="Spinosaure", type="Piscivore")
    velo = Dinsaure.objects.create(nom="Velociraptor", type="Carnivore")

    compatible, raison = verifier_compatibilite(spino, velo)

    self.assertFalse(compatible)
    self.assertIn("pas compatibles avec ce type", raison)
#
def test_dino_qui_aime_un_autre_individuellement(self):
    diplo = Dinsaure.objects.create(nom="Diplodocus", type="Herbivore")
    galli = Dinsaure.objects.create(nom="Gallimimus", type="Herbivore")
    diplo.aime.add(galli)

    compatible, raison = verifier_compatibilite(diplo, galli)

    # "aime" est l'inverse de "naime_pas" : ça REND les dinos compatibles.
    self.assertTrue(compatible)
    self.assertIn("apprécie", raison)

    # Sens 2 : même données, ordre inversé → l'autre bloc du code est testé
    compatible2, raison2 = verifier_compatibilite(galli, diplo)
    self.assertTrue(compatible2)
    self.assertIn("apprécie", raison2)

def test_naime_pas_type(self):
    trex = Dinsaure.objects.create(nom="tyranosaure", type="Carnivore",
naime_pas_types="Carnivore")
    velo = Dinsaure.objects.create(nom="Velociraptor", type="Carnivore")

    # Sens 1
    compatible, raison = verifier_compatibilite(trex, velo)
    self.assertFalse(compatible)
    self.assertIn("n'aime pas les", raison)

    # Sens 2 : même données, ordre inversé → l'autre bloc du code est testé
    compatible2, raison2 = verifier_compatibilite(velo, trex)
    self.assertFalse(compatible2)
    self.assertIn("n'aime pas les", raison2)

def test_charognard_compatible_avec_herbivore(self):
    # Un charognard (ici de type Carnivore) doit rester compatible avec
    # un herbivore, alors que la règle carnivore/herbivore les rendrait
    # normalement incompatibles.
    vautour = Dinsaure.objects.create(
        nom="Compsognathus", type="Carnivore", famille="Charognard"
    )
    tricera = Dinsaure.objects.create(nom="Tricératops", type="Herbivore")

```

```
compatible, raison = verifier_compatibilite(vautour, tricera)
self.assertTrue(compatible)
self.assertEqual(raison, "Compatible")

# Sens 2 : même données, ordre inversé
compatible2, raison2 = verifier_compatibilite(tricera, vautour)
self.assertTrue(compatible2)
self.assertEqual(raison2, "Compatible")
```